

SQL-BASED E-COMMERCE DATA ANALYSIS USING PYTHON AND SQLITE

Project Report

Submitted By

Thamindu Kavinda

Data Analytics Intern DecodeLabs

Submitted To

DecodeLabs

Submission Date

2026/05/28

1. Introduction

This project mainly revolves around SQL-based data analysis on an E-commerce Sales dataset using Python, Pandas, and SQLite in Google Colab.

The main objective of this project was to clean the dataset, store it in a database, and perform several SQL operations on it to extract valuable insights. SQL is one of the most popular tools in the field of data analytics for retrieving, selecting, grouping, and analyzing large amounts of data.

Some of the SQL operations performed in this project include SELECT, WHERE, GROUP BY, ORDER BY, COUNT, SUM, and AVG.

2. Objectives

- To load and inspect the E-Commerce dataset
- To identify and handle missing values
- To check and remove duplicate records
- To create a SQLite database from the cleaned dataset
- To perform SQL-based data analysis
- To generate business insights using SQL queries
- To improve practical knowledge of database analytics

3. Technologies Used

- Python - Data processing
- Pandas - Data manipulation
- SQLite - Database management
- SQL - Data querying and analysis
- Google Colab - Development environment
- Microsoft Excel - Data storage

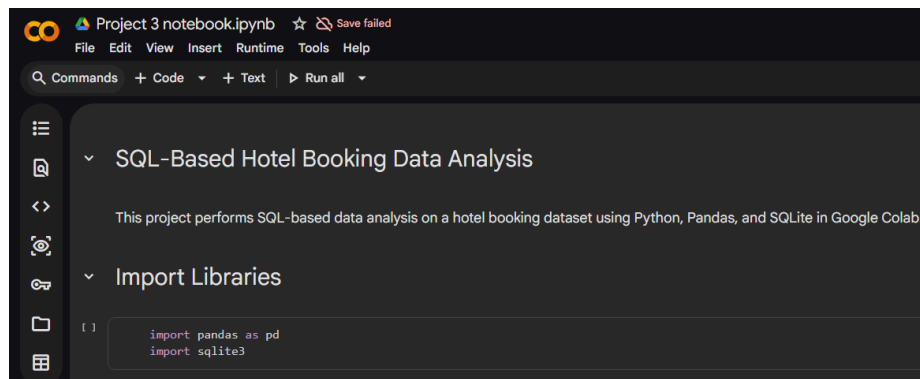
4. Dataset Description

The dataset used in this project contains E-Commerce transaction records. It includes information related to products, customer purchases, payment methods, order status, referral sources, coupon usage, and transaction values.

The dataset was first cleaned and preprocessed before being imported into a SQLite database for SQL-based analysis. The cleaned dataset was then used to perform various analytical queries and generate business insights.

5. Data Cleaning Process

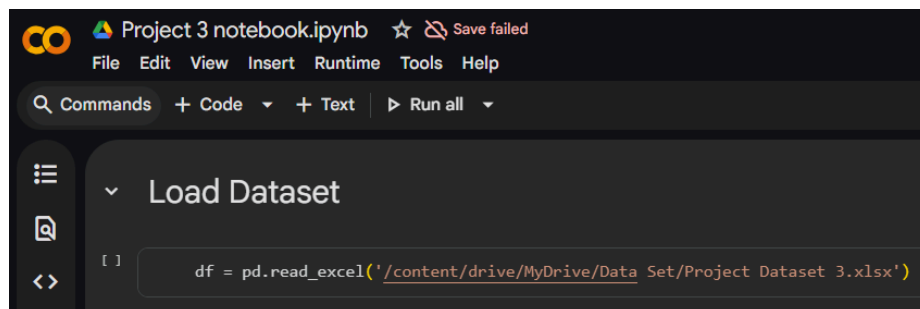
Importing Libraries



```
import pandas as pd
import sqlite3
```

The required Python libraries were imported to perform data cleaning and database operations. Pandas was used for data manipulation, while SQLite was used for database creation and SQL query execution.

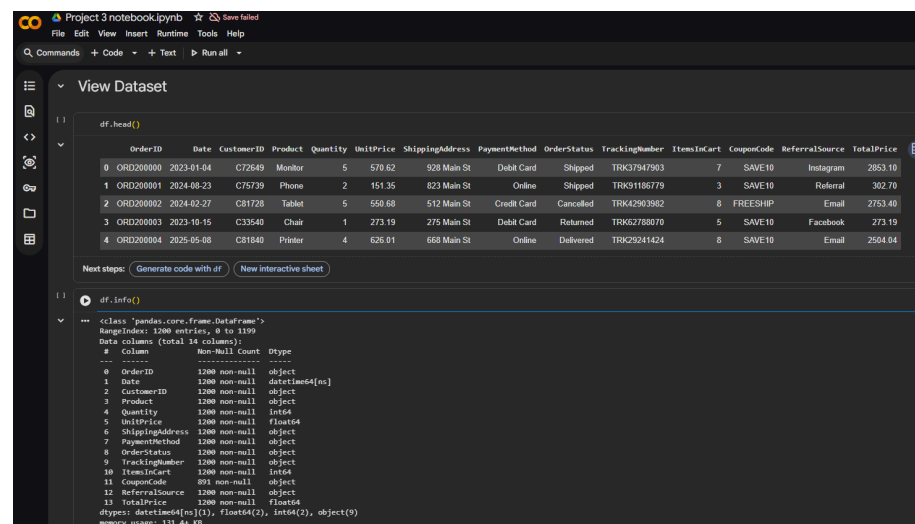
Loading the Dataset



```
df = pd.read_excel('/content/drive/MyDrive/Data Set/Project Dataset 3.xlsx')
```

The E-Commerce dataset was loaded into Google Colab using Pandas. This allowed the dataset to be inspected and prepared for further processing.

Dataset Inspection

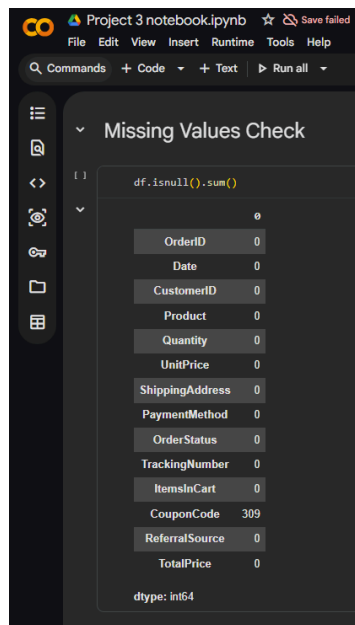


```
df.head()
df.info()
```

OrderID	Date	CustomerID	Product	Quantity	UnitPrice	ShippingAddress	PaymentMethod	OrderStatus	TrackingNumber	ItemsInCart	CouponCode	ReferralSource	TotalPrice
ORD200000	2023-01-04	C72649	Monitor	5	570.62	928 Main St	Debit Card	Shipped	TRK37947903	7	SAVE10	Instagram	2853.10
ORD200001	2024-08-23	C75739	Phone	2	151.35	823 Main St	Online	Shipped	TRK91198779	3	SAVE10	Referral	302.70
ORD200002	2024-02-27	C81728	Tablet	5	550.68	512 Main St	Credit Card	Cancelled	TRK42903982	8	FREESHIP	Email	2753.40
ORD200003	2023-10-15	C33540	Chair	1	273.19	275 Main St	Debit Card	Returned	TRK62788070	5	SAVE10	Facebook	273.19
ORD200004	2025-05-08	C81840	Printer	4	626.01	668 Main St	Online	Delivered	TRK29241424	8	SAVE10	Email	2504.04

The first few records of the dataset were displayed to understand its structure, column names, and data contents.

Missing Value Detection



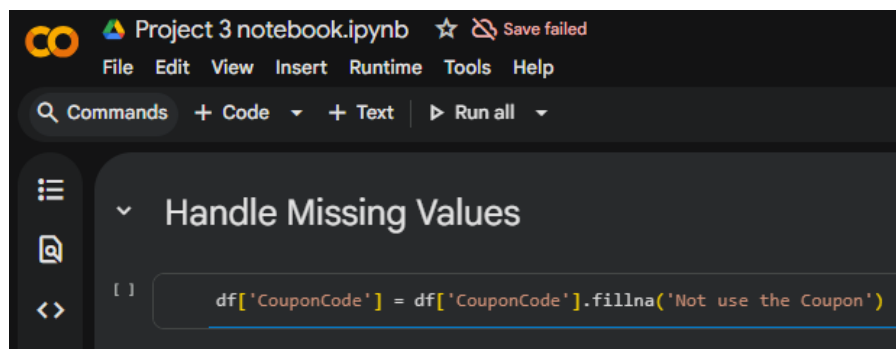
```
df.isnull().sum()
```

Column	Count
OrderID	0
Date	0
CustomerID	0
Product	0
Quantity	0
UnitPrice	0
ShippingAddress	0
PaymentMethod	0
OrderStatus	0
TrackingNumber	0
ItemsInCart	0
CouponCode	309
ReferralSource	0
TotalPrice	0

dtype: int64

The dataset was examined for missing values to identify incomplete records that could affect the accuracy of the analysis.

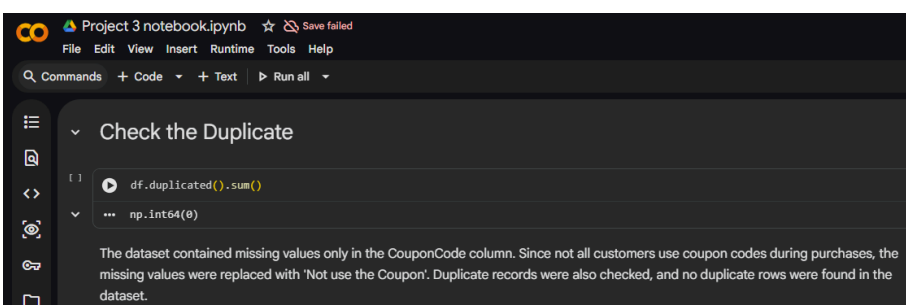
Handling Missing Values



```
df['CouponCode'] = df['CouponCode'].fillna('Not use the Coupon')
```

Missing values in the CouponCode column were replaced with an appropriate value to maintain data consistency and improve dataset quality.

Duplicate Record Checking



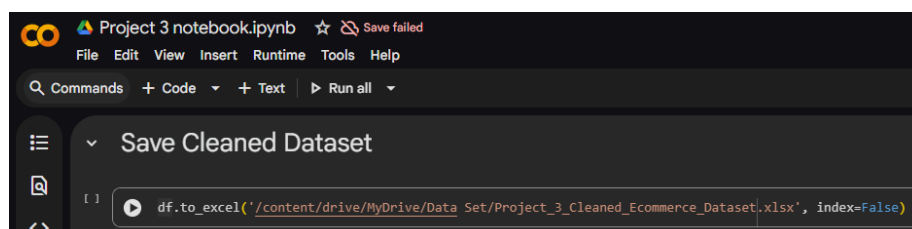
```
df.duplicated().sum()
```

np.int64(0)

The dataset contained missing values only in the CouponCode column. Since not all customers use coupon codes during purchases, the missing values were replaced with 'Not use the Coupon'. Duplicate records were also checked, and no duplicate rows were found in the dataset.

The dataset was checked for duplicate records to ensure that repeated entries would not affect the accuracy of the analytical results.

Exporting the Cleaned Dataset

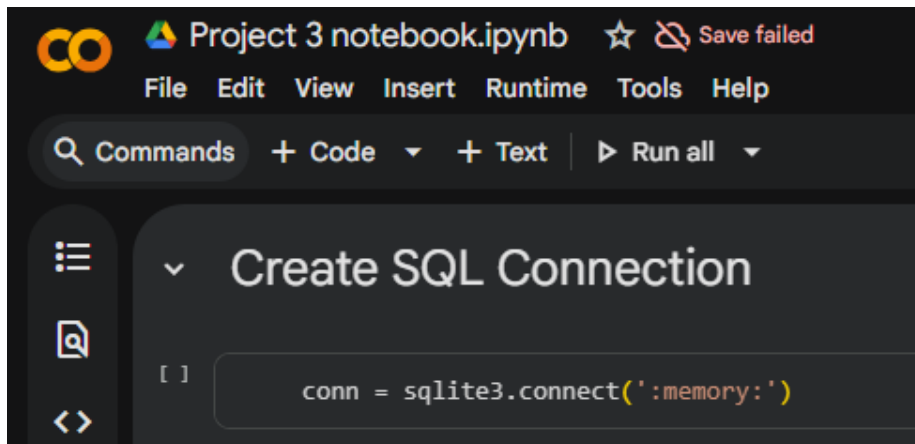


```
df.to_excel('/content/drive/MyDrive/Data Set/Project_3_Cleaned_Ecommerce_Dataset.xlsx', index=False)
```

After completing the cleaning process, the dataset was exported as a new file for future analysis and database integration.

6. Database Creation and SQL Integration

Creating SQLite Connection



The screenshot shows a Google Colab notebook titled "Project 3 notebook.ipynb". The interface includes a menu bar with "File", "Edit", "View", "Insert", "Runtime", "Tools", and "Help". Below the menu is a search bar and buttons for "Commands", "+ Code", "+ Text", and "Run all". The main code cell is titled "Create SQL Connection" and contains the following Python code:

```
[ ] conn = sqlite3.connect(':memory:')
```

An SQLite database connection was created within Google Colab. This database served as the environment for executing SQL queries on the cleaned dataset.

Creating SQL Table



The screenshot shows a Google Colab notebook titled "Project 3 notebook.ipynb". The interface includes a menu bar with "File", "Edit", "View", "Insert", "Runtime", "Tools", and "Help". Below the menu is a search bar and buttons for "Commands", "+ Code", "+ Text", and "Run all". The main code cell is titled "Convert DataFrame to SQL Table" and contains the following Python code:

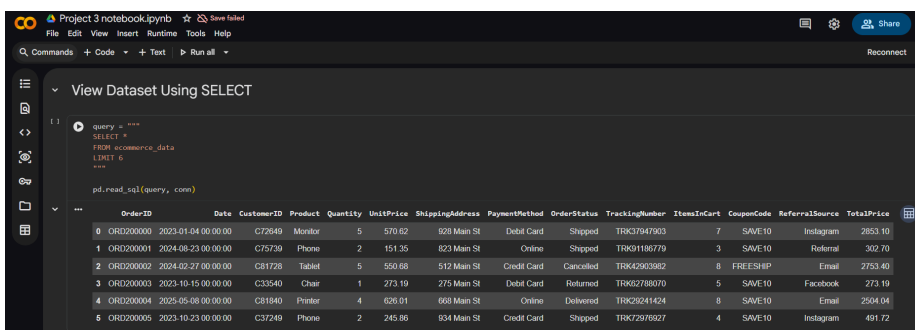
```
[ ] df.to_sql('ecommerce_data', conn, index=False, if_exists='replace')
```

Below the code cell, the output shows "1200" and a message: "The cleaned dataset was converted into an SQL table using SQLite to perform SQL-based data analysis queries."

The cleaned dataset was transferred into a database table using Pandas and SQLite integration. This allowed SQL commands to be executed directly on the dataset.

7. SQL Query Analysis

Viewing Dataset Records



The screenshot shows a Google Colab notebook titled "Project 3 notebook.ipynb". The interface includes a menu bar with "File", "Edit", "View", "Insert", "Runtime", "Tools", and "Help". Below the menu is a search bar and buttons for "Commands", "+ Code", "+ Text", and "Run all". The main code cell is titled "View Dataset Using SELECT" and contains the following Python code:

```
[ ] query = """
SELECT *
FROM ecommerce_data
LIMIT 5
"""

pd.read_sql(query, conn)
```

Below the code cell, the output shows a table with 11 columns: OrderID, Date, CustomerID, Product, Quantity, UnitPrice, ShippingAddress, PaymentMethod, OrderStatus, TrackingNumber, ItemsInCart, CouponCode, ReferralSource, and TotalPrice. The table contains 5 rows of data.

OrderID	Date	CustomerID	Product	Quantity	UnitPrice	ShippingAddress	PaymentMethod	OrderStatus	TrackingNumber	ItemsInCart	CouponCode	ReferralSource	TotalPrice	
0	ORD000000	2023-01-04 00:00:00	C72649	Monitor	5	570.62	928 Main St	Debit Card	Shipped	TRK37947903	7	SAVE10	Instagram	2853.10
1	ORD000001	2024-08-23 00:00:00	C75739	Phone	2	151.35	823 Main St	Online	Shipped	TRK91186779	3	SAVE10	Referral	302.70
2	ORD000002	2024-02-27 00:00:00	C81728	Tablet	5	550.68	512 Main St	Credit Card	Cancelled	TRK42903982	8	FREE5HP	Email	2753.40
3	ORD000003	2023-10-15 00:00:00	C33540	Chair	1	273.19	275 Main St	Debit Card	Returned	TRK602788070	5	SAVE10	Facebook	273.19
4	ORD000004	2025-05-08 00:00:00	C81840	Printer	4	626.01	668 Main St	Online	Delivered	TRK29241424	8	SAVE10	Email	2504.04
5	ORD000005	2023-10-23 00:00:00	C37249	Phone	2	245.86	934 Main St	Credit Card	Shipped	TRK72976927	4	SAVE10	Instagram	491.72

A SELECT query was executed to retrieve sample records from the database table and verify successful data insertion.

Delivered Orders Analysis

Project 3 notebook.ipynb

WHERE Clause Analysis

```
query = """
SELECT *
FROM ecommerce_data
WHERE OrderStatus = 'Delivered'
"""

pd.read_sql(query, conn)
```

	OrderID	Date	CustomerID	Product	Quantity	UnitPrice	ShippingAddress	PaymentMethod	OrderStatus	TrackingNumber	ItemsInCart	CouponCode	ReferralSource	TotalPrice
0	ORD200004	2025-05-08 00:00:00	C81840	Printer	4	626.01	668 Main St	Online	Delivered	TRK29241424	8	SAVE10	Email	2504.04
1	ORD200015	2023-07-17 00:00:00	C39416	Printer	1	473.96	942 Main St	Cash	Delivered	TRK34930938	3	Not use the Coupon	Google	473.96
2	ORD200021	2024-11-17 00:00:00	C99023	Monitor	4	342.95	569 Main St	Cash	Delivered	TRK32646983	6	Not use the Coupon	Google	1371.80
3	ORD200024	2024-10-30 00:00:00	C17470	Laptop	2	230.95	956 Main St	Online	Delivered	TRK38775057	3	WINTER15	Referral	461.90
4	ORD200025	2024-07-24 00:00:00	C42577	Monitor	1	359.98	709 Main St	Debit Card	Delivered	TRK629423327	3	WINTER15	Email	359.98
226	ORD201165	2023-07-31 00:00:00	C95558	Chair	3	324.47	971 Main St	Cash	Delivered	TRK37075292	4	WINTER15	Email	973.41
227	ORD201168	2023-04-08 00:00:00	C89559	Laptop	2	331.42	297 Main St	Credit Card	Delivered	TRK52338108	6	WINTER15	Google	662.84
228	ORD201171	2023-06-28 00:00:00	C39911	Phone	4	516.40	699 Main St	Credit Card	Delivered	TRK380431998	7	SAVE10	Facebook	2073.60
229	ORD201187	2024-03-13 00:00:00	C26340	Desk	2	184.60	562 Main St	Gift Card	Delivered	TRK39962703	7	WINTER15	Email	369.20
230	ORD201197	2023-07-13 00:00:00	C79674	Tablet	2	436.84	275 Main St	Online	Delivered	TRK360303056	2	FREESHIP	Instagram	873.68

231 rows x 14 columns

A WHERE clause was used to retrieve only delivered orders. This analysis helped identify successfully completed transactions within the dataset.

Highest Value Products

Project 3 notebook.ipynb

ORDER BY Analysis

```
query = """
SELECT Product, TotalPrice
FROM ecommerce_data
ORDER BY TotalPrice DESC
LIMIT 10
"""

pd.read_sql(query, conn)
```

	Product	TotalPrice
0	Tablet	3456.40
1	Monitor	3390.95
2	Laptop	3390.80
3	Chair	3384.90
4	Tablet	3370.20
5	Printer	3353.75
6	Laptop	3352.40
7	Printer	3334.00
8	Phone	3322.55
9	Laptop	3313.90

An ORDER BY query was used to sort products based on transaction value. This helped identify products generating the highest sales revenue.

Product Order Analysis

Project 3 notebook.ipynb

GROUP BY Analysis

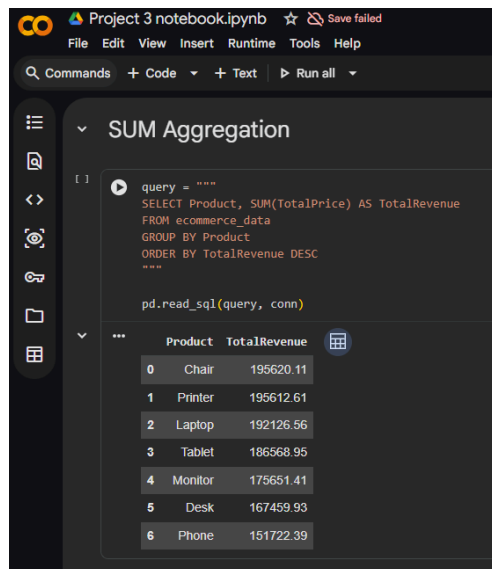
```
query = """
SELECT Product, COUNT(*) AS TotalOrders
FROM ecommerce_data
GROUP BY Product
"""

pd.read_sql(query, conn)
```

	Product	TotalOrders
0	Chair	178
1	Desk	170
2	Laptop	173
3	Monitor	163
4	Phone	156
5	Printer	181
6	Tablet	179

A GROUP BY query was performed to count the number of orders associated with each product. This helped identify the most popular products among customers.

Revenue Analysis



Project 3 notebook.ipynb

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

SUM Aggregation

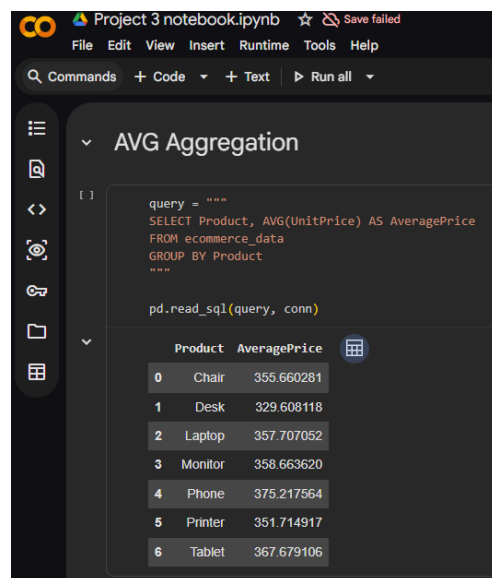
```
query = """
SELECT Product, SUM(TotalPrice) AS TotalRevenue
FROM ecommerce_data
GROUP BY Product
ORDER BY TotalRevenue DESC
"""

pd.read_sql(query, conn)
```

	Product	TotalRevenue
0	Chair	195620.11
1	Printer	195612.61
2	Laptop	192126.56
3	Tablet	186568.95
4	Monitor	175651.41
5	Desk	167459.93
6	Phone	151722.39

The SUM function was used to calculate total revenue generated by each product. This analysis highlighted the products contributing most to overall sales.

Average Product Price Analysis



Project 3 notebook.ipynb

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

AVG Aggregation

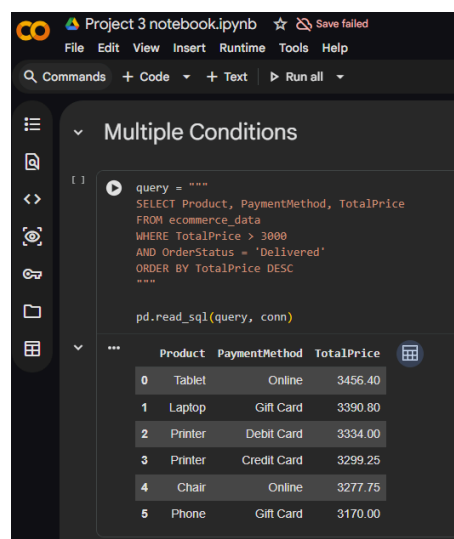
```
query = """
SELECT Product, AVG(UnitPrice) AS AveragePrice
FROM ecommerce_data
GROUP BY Product
"""

pd.read_sql(query, conn)
```

	Product	AveragePrice
0	Chair	355.660281
1	Desk	329.608118
2	Laptop	357.707052
3	Monitor	358.663620
4	Phone	375.217564
5	Printer	351.714917
6	Tablet	367.679106

The AVG function was used to calculate the average price of products in the dataset. This provided insights into product pricing patterns.

High-Value Delivered Orders



Project 3 notebook.ipynb

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

Multiple Conditions

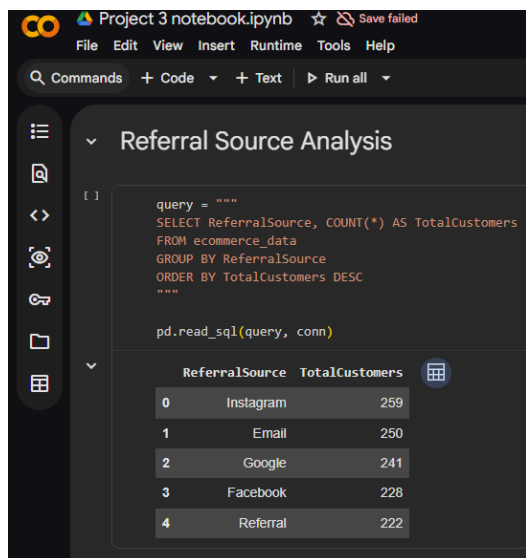
```
query = """
SELECT Product, PaymentMethod, TotalPrice
FROM ecommerce_data
WHERE TotalPrice > 3000
AND OrderStatus = 'Delivered'
ORDER BY TotalPrice DESC
"""

pd.read_sql(query, conn)
```

	Product	PaymentMethod	TotalPrice
0	Tablet	Online	3456.40
1	Laptop	Gift Card	3390.80
2	Printer	Debit Card	3334.00
3	Printer	Credit Card	3299.25
4	Chair	Online	3277.75
5	Phone	Gift Card	3170.00

Multiple SQL conditions were used to identify delivered orders with high transaction values. This analysis helped understand premium customer purchases.

Referral Source Analysis



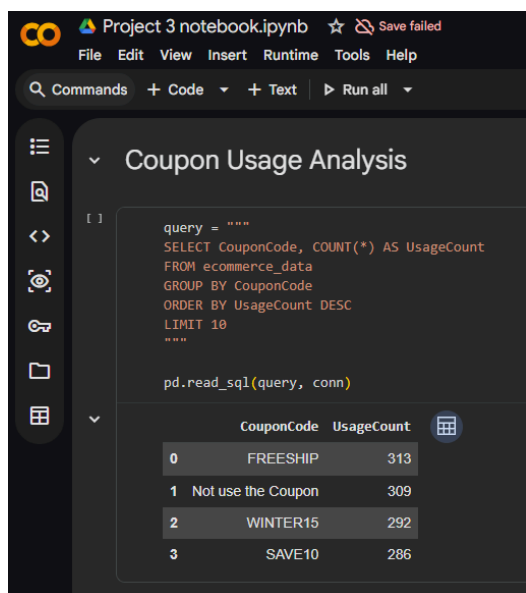
```
query = """
SELECT ReferralSource, COUNT(*) AS TotalCustomers
FROM ecommerce_data
GROUP BY ReferralSource
ORDER BY TotalCustomers DESC
"""

pd.read_sql(query, conn)
```

	ReferralSource	TotalCustomers
0	Instagram	259
1	Email	250
2	Google	241
3	Facebook	228
4	Referral	222

The dataset was analyzed based on referral sources to determine which marketing channels generated the highest customer engagement and transactions.

Coupon Usage Analysis



```
query = """
SELECT CouponCode, COUNT(*) AS UsageCount
FROM ecommerce_data
GROUP BY CouponCode
ORDER BY UsageCount DESC
LIMIT 10
"""

pd.read_sql(query, conn)
```

	CouponCode	UsageCount
0	FREESHIP	313
1	Not use the Coupon	309
2	WINTER15	292
3	SAVE10	286

Coupon usage data was analyzed to evaluate customer participation in promotional campaigns and discount programs.

9. Conclusion

This project helped me learn about the practical application of SQL in data analysis. It was possible to do this by using Python, Pandas, and SQLite to clean, structure, and analyze an e-commerce dataset.

One of the goals of the project was to learn about the theoretical and practical aspects related to databases, SQL queries, and data analysis. The need for SQL knowledge was also evident from this project.